



TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

TÉCNICA DE PROGRAMAÇÃO I
REVISÃO



Técnica de Programação I
Profº Luiz Cláudio

Java - Operadores Aritméticos



Função	Sinal
Adição	+
Subtração	-
Multiplicação	*
Divisão	/
Resto da divisão	%
Incremento	++
Decremento	--

Java - Operadores Relacionais



Função	Sinal
Igual	==
Diferente	!=
Maior que	>
Maior ou igual a	>=
Menor que	<
Menor ou igual a	<=

Java - Conversão de Tipos



Converter em	y recebe o valor convertido
int	<code>int y = Integer.parseInt(x)</code>
float	<code>float y = Float.parseFloat(x)</code>
double	<code>double y = Double.parseDouble(x)</code>

Entrada de Dados pelo Teclado

```
Scanner entrada = new Scanner(System.in);
```

```
System.out.println("Digite o primeiro numero:");  
numero = entrada.nextInt();
```

Comando Escreva

Comando Leia

Atenção: Para funcionar o Scanner devemos importar a biblioteca do Scanner de leitura

No Java utilizando o comando
`import java.util.Scanner;`

Exemplo Entrada de Dados pelo Teclado

```
int num1,num2;    String nome;
double num3,resultado;
Scanner entrada = new Scanner(System.in);
System.out.println("Digite o nome:");
nome = entrada.next();
System.out.println("Digite o primeiro numero:");
num1 = entrada.nextInt();
System.out.println("Digite o segundo numero:");
num2 = entrada.nextInt();
System.out.println("Digite o terceiro numero:");
num3 = entrada.nextDouble();
resultado = num1 + num2+ num3;
System.out.println("Resultado: " + resultado);
```

Orientação Objetos



Na POO o programador é responsável por moldar o mundo dos objetos, e explicar para estes objetos como eles devem interagir entre si.

CLASSE



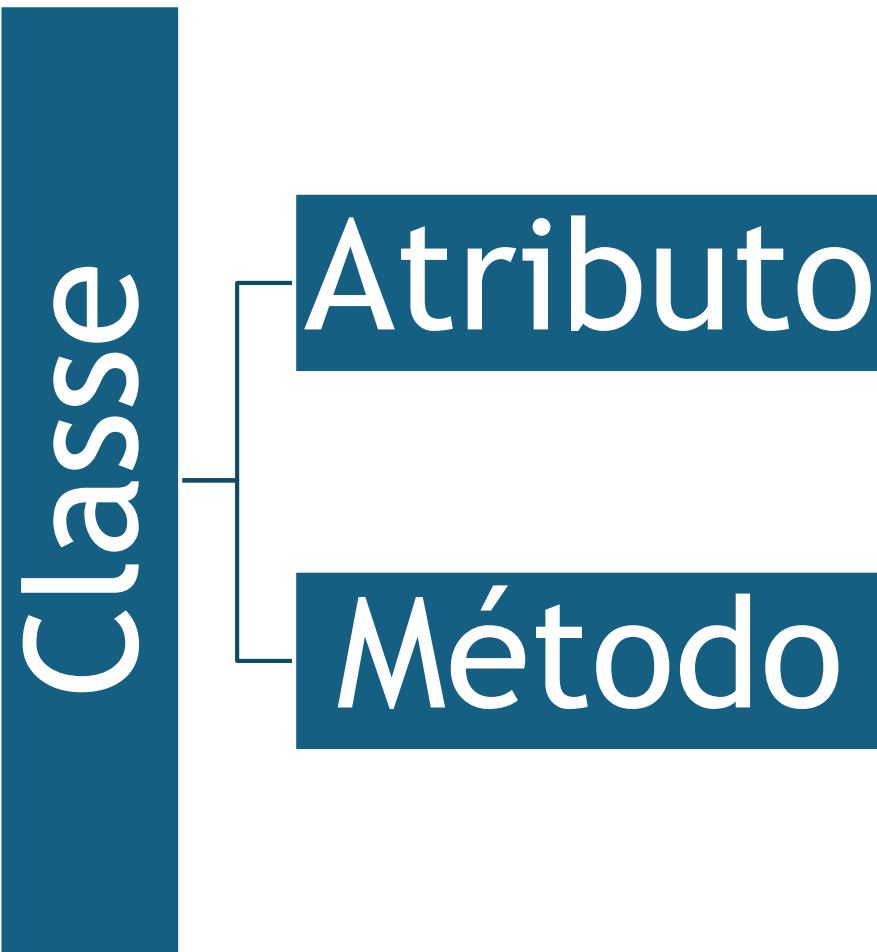
Uma classe funciona como uma “receita” para criar objetos. Inclusive, vários objetos podem ser criados a partir de uma única classe.



Casas construídas a partir da mesma planta podem possuir características diferentes. Por exemplo, a cor das paredes.



CLASSE



ATRIBUTOS



Atributo

É um dado para o qual cada objeto tem seu próprio valor.

Definem características de objetos.

Exemplo de atributos:

Marca

Velocidade

Cor

Portas



MÉTODOS



Método

Definem as habilidades dos objetos.

Ações que os objetos podem executar.

Métodos são declarados dentro de uma classe para representar as operações que os objetos pertencentes a esta classe podem executar.

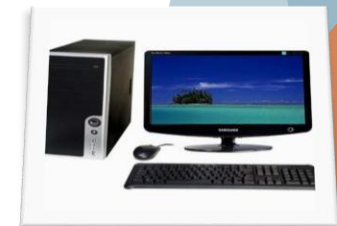




OBJETO



Uma aplicação orientada a objetos é composta por objetos.
É uma Instância(criação) de uma classe
Podemos definir os objetos como sendo os objetos físicos do mundo real (carro, avião, cachorro, casa, telefone, computador).



Orientação Objetos em Java



► Assinatura de método

◦ Formado por:

- nome do método.
- argumentos (parâmetros)
- tipo de resposta (retorno)

Parâmetros: São valores (como variáveis) atribuídos aos métodos para alterar o seu comportamento em tempo de execução.



```
public String calculaSalario(int valor)
```



Retorno



Nome



Parâmetro entrada

Orientação Objetos em Java



- ▶ Assinatura de método sem parâmetros e sem retorno
 - Formado por:
 - nome do método.
 - argumentos (parâmetros)
 - tipo de resposta (retorno)

```
public void calculaSalario()
```

Retorno vazio

Nome

Sem parâmetros



CLASSE CARRO



Carro

- marca : String
- velocidade : double
- cor : String
- porta: int

- +cadastrarDadosCarro(): void
- +mostrarDadosCarro(): void
- +alterarDadosCarro()
- +aumentarVelocidade(int v):void



Modificadores de acesso



► Modificadores de acesso

► Private

- Só fica visível dentro da classe em que foi implementado

► Public

- Fica visível em toda a aplicação

► Protected

- Fica visível na classe em que foi implementado e em suas sub-classe

ENCAPSULAMENTO



- ▶ Consiste na proteção dos atributos de uma classe (e posteriormente dos objetos) de acessos externos.
- ▶ Considerando que todas as regras referentes a classe estão contidas na própria classe (e nunca em outra parte da aplicação)

CLASSE MAIN



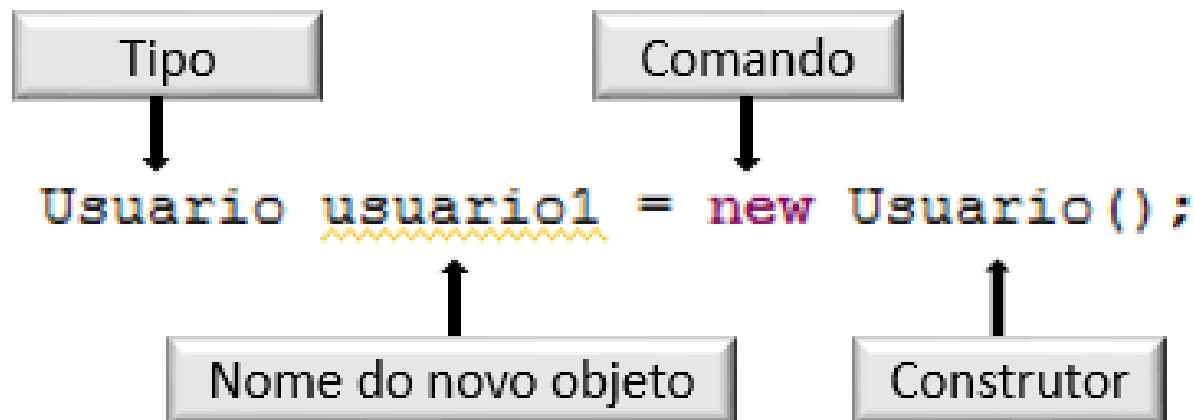
- ▶ A classe principal será usada somente para conter o método main

Principal
<u>+ main(args[] : String) : void</u>

COMO INSTANCIAR UM OBJETO



```
public class Principal {  
  
    public static void main(String[] args) {
```



Codificação da classe Usuario

```
public class Usuario {  
  
    // Atributos  
    private String nome;  
    private String email;  
    private String login;  
    private String senha;  
  
    // Construtores  
    // Inicializa os atributos vazios  
    public Usuario() {  
        this("", "", "", "");  
    }  
    // Inicializa os atributos com valores passados por parametro  
    public Usuario(String email, String login, String nome, String senha) {  
        this.email = email;  
        this.login = login;  
        this.nome = nome;  
        this.senha = senha;  
    }  
  
    // Getters e Setters  
    public String getNome() {  
        return nome;  
    }  
  
    public void setNome(String nome) {  
        this.nome = nome;  
    }  
  
    // Implementação dos demais getters e setters  
  
    // Métodos específicos da classe  
    public void provarExistencia(){  
        System.out.println("Oi, eu existo!");  
    }  
}
```

Codificação da classe Principal

```
public class Principal {  
  
    public static void main(String[] args) {  
  
        Usuario usuario1 = new Usuario();  
  
        usuario1.provarExistencia();  
  
    }  
}
```

Criando o Primeiro Programa

Defina o Nome do Projeto e Project Location , defina o caminho onde será armazenado o projeto, em seguida clique em finish

New Java Application

Steps

1. Choose Project
2. **Name and Location**

Name and Location

Project Name: PrimeiroProjetoOO

Project Location: C:\Users\lcb_8\Documents\NetBeansProjects Browse...

Project Folder: rs\lcb_8\Documents\NetBeansProjects\PrimeiroProjetoOO

Artifact Id: PrimeiroProjetoOO

Group Id: com.mycompany

Version: 1.0-SNAPSHOT

Package: com.mycompany.primeiroprojeto00 (Optional)

< Back Next > Finish Cancel Help

Criação de uma classe



Para Criar uma nova classe, clique com o botão direito no nome do projeto , New-> Java Class...

Defina um classe com nome Usuario, em package selecione o pacote padrão do projeto, depois clique em Finalizar

The screenshot illustrates the process of creating a new Java class in an IDE. The 'New' menu is open, and 'Java Class...' is selected. The 'Name and Location' dialog is shown with the following details:

- Class Name:** Usuario
- Project:** PrimeiroProjetoOO
- Location:** Source Packages
- Package:** com.mycompany.primeiroprojeto00 (selected)
- Created File:** C:\Users\user\Documents\PrimeiroProjetoOO\src\main\java\com\mycompany\primeiroprojeto00\Usuario.java
- Superclass:** (empty)
- Interfaces:** (empty)
- Warning:** It is highly recommended that you do not place Java classes in the default package.
- Buttons:** < Back, Next >, Finish (highlighted), Cancel, Help

Criação da estrutura da classe de modelagem



Construtores gerados

```
public class Usuario {  
    private String nome;  
    private int idade;  
    private String email;  
    private String telefone;  
}
```

← Criar os
atributos na
classe

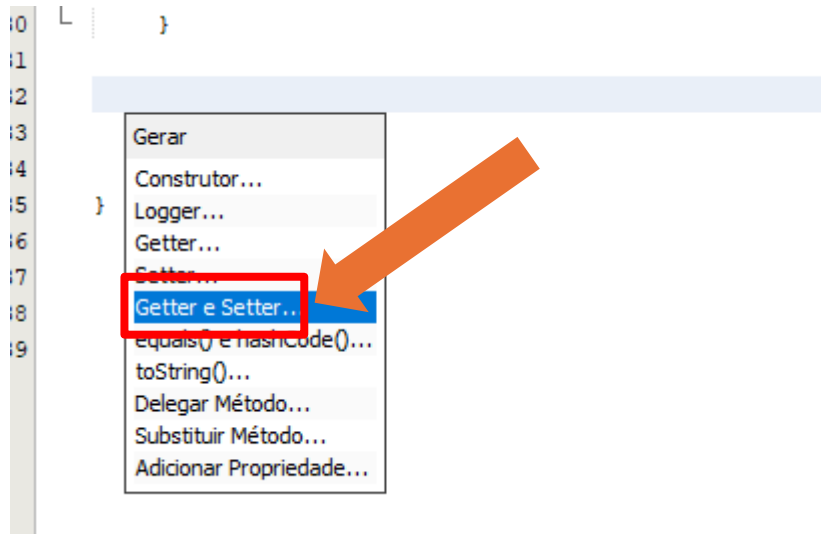
Criação da estrutura da classe de modelagem – Getters e Setters



Encapsulamento/getters e setters gerados

Geração de getters e setters

Clique com o botão direito abaixo dos atributos em insert code.. -> -Getter e Setter



```
public String getNome() {  
    return nome;  
}  
  
public void setNome(String nome) {  
    this.nome = nome;  
}  
  
public int getIdade() {  
    return idade;  
}  
  
public void setIdade(int idade) {  
    this.idade = idade;  
}  
  
public String getEmail() {  
    return email;  
}  
  
public void setEmail(String email) {  
    this.email = email;  
}  
  
public String getTelefone() {  
    return telefone;  
}  
  
public void setTelefone(String telefone) {  
    this.telefone = telefone;  
}
```


Criando Um método Mensagem-Classe Usuario



Implementar o método Mensagem,
abaixo dos atributos encapsulado

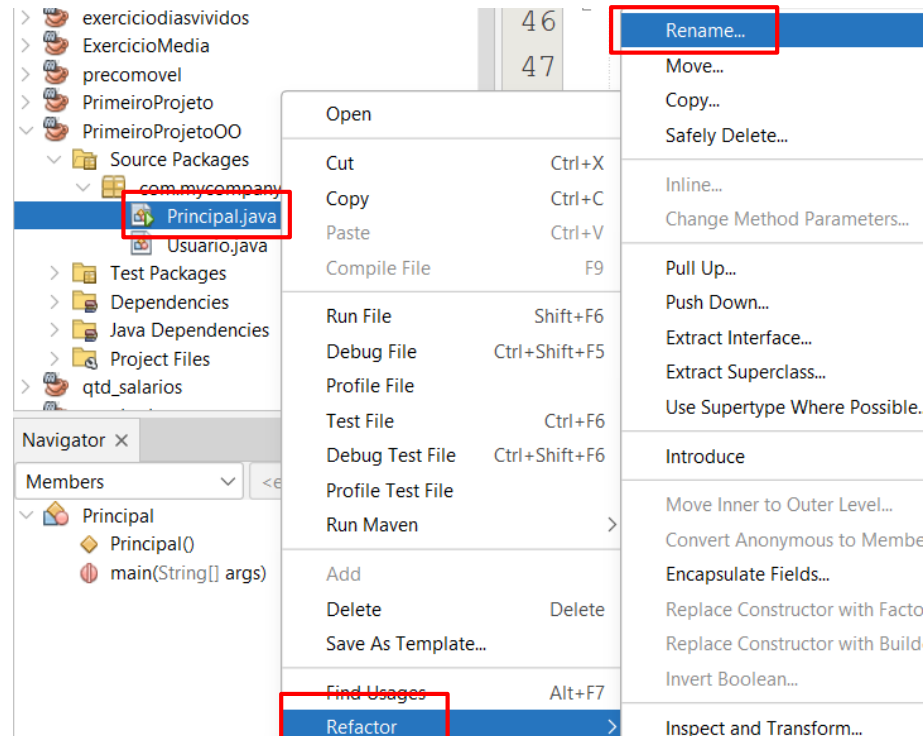
```
public void setTelefone(String telefone) {  
    this.telefone = telefone;  
}
```

```
public void Mensagem() {  
    System.out.println("Bem vindo ao Java Orientado a Objetos");  
}
```

Criando a Classe Principal



Renomei a Classe Principal que foi criado pelo Projeto , para o nome classe Principal



Criando a Classe Principal-Instanciando um Objeto



Criamos objeto usuario1 , que possui todos os métodos definidos na classe Usuario , este é objeto é instanciado dentro do método Principal(main)

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    //Instaciando um objeto  
    Usuario usuario1 = new Usuario();  
}
```

Para chamar um método de uma classe , colocamos o nome do objeto , seguido do nome do método

Usuario1.Mensagem

```
public static void main(String[] args) {  
    // TODO code application logic here  
  
    //Instaciando um objeto  
    Usuario usuario1 = new Usuario();  
    usuario1.Mensagem();  
}
```

Executando um Projeto em Java



O painel console simula um prompt em ambiente texto

A screenshot of an IDE's console window. The title bar reads "Output - Run (Principal) X". The console output shows the following lines: "Nothing to compile - all classes are up to date", "--- exec:3.1.0:exec (default-cli) @ PrimeiroProjetoOO ---", and "Bem vindo ao Java Orientado a Objetos". The last line is highlighted with a red rectangular box. Below this, a dashed line separates the output from the status "BUILD SUCCESS" in green text. On the left side of the console, there are several icons: a green play button, a yellow play button, a magnifying glass, and a document icon. A black arrow points from the text below to the magnifying glass icon.

```
Output - Run (Principal) X
Nothing to compile - all classes are up to date
--- exec:3.1.0:exec (default-cli) @ PrimeiroProjetoOO ---
Bem vindo ao Java Orientado a Objetos
-----
BUILD SUCCESS
```

Enfim, a emocionante manifestação de vida do objeto usuario1

Incrementando a Aplicação



Vamos acrescentar na classe Principal, armazenando valores nos atributos da classe Usuário, para guardar valores nos atributos , utilizamos a função set.

Classe Principal: Armazenando dados nos atributos, para enviar a classe Usuário

```
public static void main(String[] args) {  
  
    Usuario usu = new Usuario();  
  
    usu.setNome("Luiz");  
    usu.setEmail("luiz83@hotmail");  
    usu.setTelefone("99323-32234");  
    usu.setIdade(10);  
    usu.Mensagem();  
}
```

Incrementando a Aplicação



Inserindo o conteúdo dos atributos na classe Usuario, dentro do método Mensagem, para resgatar os valores dos atributos utilizamos a função get

Classe Usuario: Alterar o método Mensagem

```
public void Mensagem() {  
    System.out.println("Nome: " + getNome() + "\n" +  
        "Email: " + getEmail() + "\n" +  
        "Telefone: " + getTelefone() + "\n" +  
        "Idade: " + getIdade());  
}
```

Executando novamente Projeto em Java



Clique na classe Principal e Pressione shift+ F6 e veja o resultado:

```
Output - Run (Principal) X
Changes detected - recompiling the module! :source
Compiling 2 source files with javac [debug target 21] to
--- exec:3.1.0:exec (default-cli) @ PrimeiroProjetoOO ---
Nome: Luiz
Email: luiz83@hotmail
    Telefone: 99323-32234
Idade: 10
BUILD SUCCESS
```

Executando novamente Projeto em Java



Para ler valores e passar para a classe Usuario:

Vamos acrescentar na classe Principal, armazenando valores nos atributos da classe Usuário, para guardar valores nos atributos, utilizamos a função set. Porém agora vamos fazer a leitura dos dados, utilizando a biblioteca Scanner.

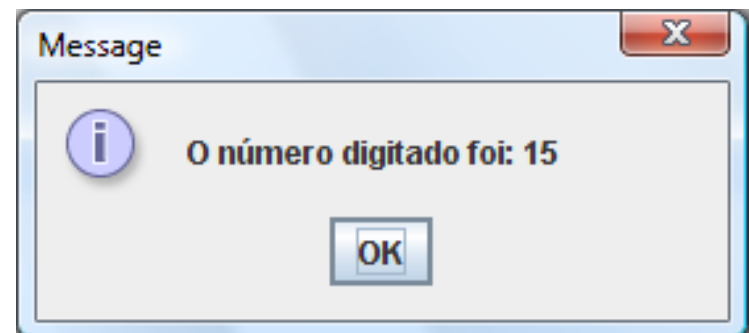
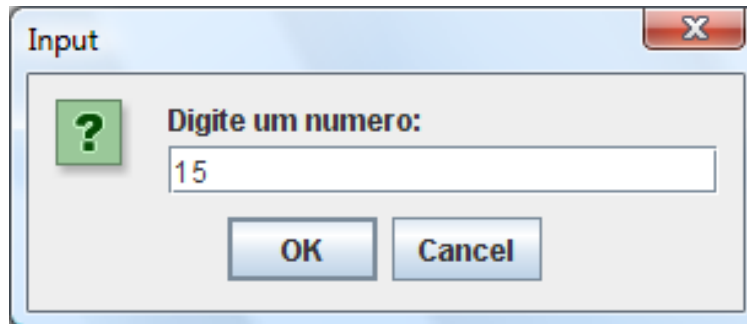
```
public static void main(String[] args) {  
  
    Usuario usu = new Usuario();  
    Scanner leia = new Scanner(System.in);  
    System.out.println("Digite o nome: ");  
    usu.setNome(leia.nextLine());  
    System.out.println("Digite o e-mail: ");  
    usu.setEmail(leia.next());  
    System.out.println("Digite o Telefone: ");  
    usu.setTelefone(leia.next());  
    System.out.println("Digite a idade: ");  
    usu.setIdade(leia.nextInt());  
    usu.Mensagem();  
  
}
```


Java - Entrada e Saída de dados



- Classe JOptionPane e a diretiva import.

```
import javax.swing.JOptionPane;  
// ou import javax.swing.*;  
  
public class TesteEntradaSaida {  
  
    public static void main(String[] args) {  
  
        String numero = JOptionPane.showInputDialog("Digite um numero:");  
  
        JOptionPane.showMessageDialog(null, "O número digitado foi: " + numero);  
  
    }  
}
```



Java - ESTRUTURAS CONDICIONAIS IF / CASE



```
if (num1 >= 10) {  
    System.out.println("Condição verdadeira!");  
} else {  
    System.out.println("Condição falsa!");  
}
```

```
switch (op) {  
    case 1:  
        System.out.println("Caso op igual a 1...");  
        break;  
    case 2:  
        System.out.println("Caso op igual a 2...");  
        break;  
    case 3:  
        System.out.println("Caso op igual a 3...");  
        break;  
    default:  
        System.out.println("Caso op não seja 1, 2 ou 3");  
        break;  
}
```

Operadores Lógicos

Estrutura Condicional

O uso do operador **&&** (e) as duas condições devem ser verdadeira

```
if ( (expressao_logica) && (expressão_logica))
```

```
{
```

```
/* Bloco de comandos */
```

```
}
```

```
else if ( (expressao_logica) && (expressão_logica))
```

```
{
```

```
/* Bloco de comandos */ }
```

Exemplo uso operador Lógico

Exemplo:

```
if ((nota > 0) && ( nota <= 5))  
{  
    System.out.println("Reprovado");  
}  
else  
{  
    System.out.println("Aprovado");  
}
```

Se a condição for verdadeira

Senão a condição não for verdadeira

Exemplo comando if operador lógico and

```
setMedia(Double.parseDouble(  
JOptionPane.showInputDialog("Digite a média:"))));
```

```
If ( (media > 0) &&(media <= 5)){  
    System.out.println("Reprovado");  
    JOptionPane.showMessageDialog(null, "Reprovado");  
}  
Else if (( media > 5) && (media <=10))  
{  
    System.out.println("Aprovado");  
    JOptionPane.showMessageDialog(null, "Aprovado");  
}
```

Exemplo comando if operador lógico OR

```
setMedia(Double.parseDouble(  
JOptionPane.showInputDialog("Digite a média:")));
```

```
If ( (media == 6) || (media == 7))
```

```
    System.out.println("Aprovado");
```

```
{JOptionPane.showMessageDialog(null,"Aprovado");
```

```
}
```

```
Else if (( media == 5) || (media < 5)){
```

```
    System.out.println("Reprovado");
```

```
JOptionPane.showMessageDialog(null,"Reprovado");}
```

Java - conversão de valores e leitura de dados



```
public void entrar(){  
    // Lê um valor, converte de String para double e atribui a variável valor  
    double valor = Double.parseDouble(JOptionPane.showInputDialog("Digite o valor da entrada: "));  
}
```

Menu de Opções no Java



```
Caixa cx1 = new Caixa(); // Instanciação do objeto cx1
int op; // declaração da variável de opções
do{ // Início do looping do-while
    // Apresentação e leitura do menu de opções
    op = Integer.parseInt(JOptionPane.showInputDialog("Digite: \n1 - Entrada "
        "\n2 - Retirada \n3 - Consultar saldo \n0 - Sair "));
    switch (op) { // Abertura da estrutura de switch-case
        case 1:
            cx1.entrar(); // Chamada ao método entrar do objeto cx1
            break;
        case 2:
            cx1.retirar(); // Chamada ao método retirar do objeto cx1
            break;
        case 3:
            // Apresentação do conteúdo do atributo saldo
            JOptionPane.showMessageDialog(null, "Saldo atual: " + cx1.getSaldo());
            break;
        case 0:
            JOptionPane.showMessageDialog(null, "Finalizando programa!");
            break;
        default:
            JOptionPane.showMessageDialog(null, "Opção inválida!");
    }
}while(op != 0); // Repetirá as operações enquanto a opção for diferente de zero
```




- Os componentes GUI Swing estão dentro do pacote **javax.swing** que são utilizados para construir as interfaces gráficas.

Controles Swing

label Rótulo

OK Button

DN Botão Alternar

☐ Caixa de seleção

☐ Caixa de combinação

☐ Lista

AWT

A Rótulo

ab Campo de texto

☑ Caixa de seleção

☐ Lista

☐ Painel de rolagem

☐ Canvas

☐ Menu pop-up

OK Button

☐ Área de texto

☐ Opções

☐ Barra de rolagem

☐ Painel

☐ Barra de menu

TÉCNICA DE PROGRAMAÇÃO I

EXERCÍCIOS DE REVISÃO



Exercício1



O programa deve ser digitado o ano que a pessoa nasceu e o ano atual e o nome da pessoa, calcular a idade e mostrar

Idade

-anoAtual: int

-anoNasc: int

-idade: int

-nome: String

+cadastrardados()

+calcularIdade()

+mostrarDados():void

Principal

+ main(args{}: String): void

Exercício 2

CalculoAreaTriangulo



Triangulo

- area: double
 - base: double
 - altura: double
- + inserirDadosTriangulo()
+ calcularArea()
+ mostrarAreaTriangulo(): double

Classe: Principal

Método main

- Instanciar um objeto do tipo Triangulo chamado triang
- Apresentar um menu com as opções:
 - 1- Inserir dados triangulo
 - 2- calcular área do triangulo
 - 3- mostrar área triangulo
 - 0- Sair

Classe: Triangulo

Métodos	InserirDadosTriangulo: faça o input dos atributos altura e base do triângulo
	calcularArea: $area = (base * altura) / 2$.
	mostrarAreaTriangulo: Apresentar (showMessageDialog) com área do triângulo